Software Requirements Specification

Secure Password Manager — Browser Extension

Version 0.1.0 | Phase-1 (Demo Mode)

Standard: IEEE 830-1998

Date: 2026-04-04

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture
4. UML Diagrams
5. Functional Requirements
6. Non-Functional Requirements
7. External Interface Requirements
8. Data Dictionary
9. Security Requirements
10. Appendix

---

1. Introduction

1.1 Purpose

This SRS defines the software requirements for the **Secure Password Manager**, a zero-plaintext Firefox browser extension that captures, encrypts, stores, and auto-fills user credentials. The system employs multi-layer encryption (AES-256-GCM, PBKDF2, SHA-256) to ensure that no credential is ever stored or transmitted in plaintext.

1.2 Scope

Attribute	Detail
Product Name	Secure Password Manager
Type	Firefox WebExtension (Manifest V2)
Version	0.1.0
Phase	Phase-1 (Internal Demo)
Target Browser	Firefox 78.0+
Backend	Node.js HTTP Server + MongoDB
Deployment	Local development (localhost:8080)

The system consists of four major subsystems:

1. **Browser Extension** (popup UI, background service, content scripts)
2. **Cryptographic Engine** (client-side AES-GCM / PBKDF2 / SHA-256)
3. **Backend API Server** (Node.js REST API — the "CPU")
4. **SSO Web Portal** (supplementary login portal)

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
CPU	Central Processing Unit — the project's name for the backend server
Vault	Encrypted credential storage (local + cloud)
Master Password	The single password that derives all encryption keys
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode
PBKDF2	Password-Based Key Derivation Function 2
Blind Index	SHA-256 hash of email used for lookup without revealing the email
SSO	Single Sign-On

Metadata Encryption	AES-GCM encryption using a static internal key for usernames, emails, timestamps
Dynamic Key	A per-credential AES key derived from master password + username + hour + counter via PBKDF2

1.4 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- Web Crypto API Specification (W3C)
- WebExtensions API Documentation (MDN)
- MongoDB Driver Documentation (v6.x)

---

2. Overall Description

2.1 Product Perspective

The Secure Password Manager operates as a Firefox browser extension with a companion Node.js backend. It intercepts login form submissions on visited websites, captures credentials, encrypts them client-side, and stores encrypted packets in MongoDB via the backend API. All cryptographic operations occur in the browser's Web Crypto API — the server **never** sees plaintext passwords.

2.2 Product Functions (Summary)

#	Function	Description
F1	**User Registration**	Register with username, email, master password, and safety Q&A
F2	**Email Verification**	Verify email via a tokenized link
F3	**User Login / Vault Unlock**	Authenticate and load master password into session memory
F4	**Credential Capture**	Intercept form submissions and capture username/password pairs
F5	**Credential Save**	Encrypt and store captured credentials to the vault
F6	**Vault Browse**	List and decrypt saved credentials on demand
F7	**Auto-Fill**	Automatically fill login forms on matching sites
F8	**Account Recovery**	Recover account via safety question/answer
F9	**Password Reset**	Reset master password after recovery verification
F10	**SSO Code Issuance**	Issue single-sign-on codes via the web portal
F11	**Session Management**	JWT-like token sessions with expiry
F12	**Cloud Sync**	Optional hashed cloud backup of vault entries

2.3 User Characteristics

User Class	Description
End User	Individual managing personal web credentials; requires no technical knowledge
Administrator	Developer/operator managing the backend server and MongoDB instance

2.4 Constraints

- Firefox 78.0+ only (Manifest V2, `browser.*` API namespace)
- Backend must run on `localhost:8080` during Phase-1
- Demo mode uses in-memory mock collections (no persistent MongoDB in Phase-1)

- Master password is held in RAM only during an active session
- SMTP configuration required for email verification in production

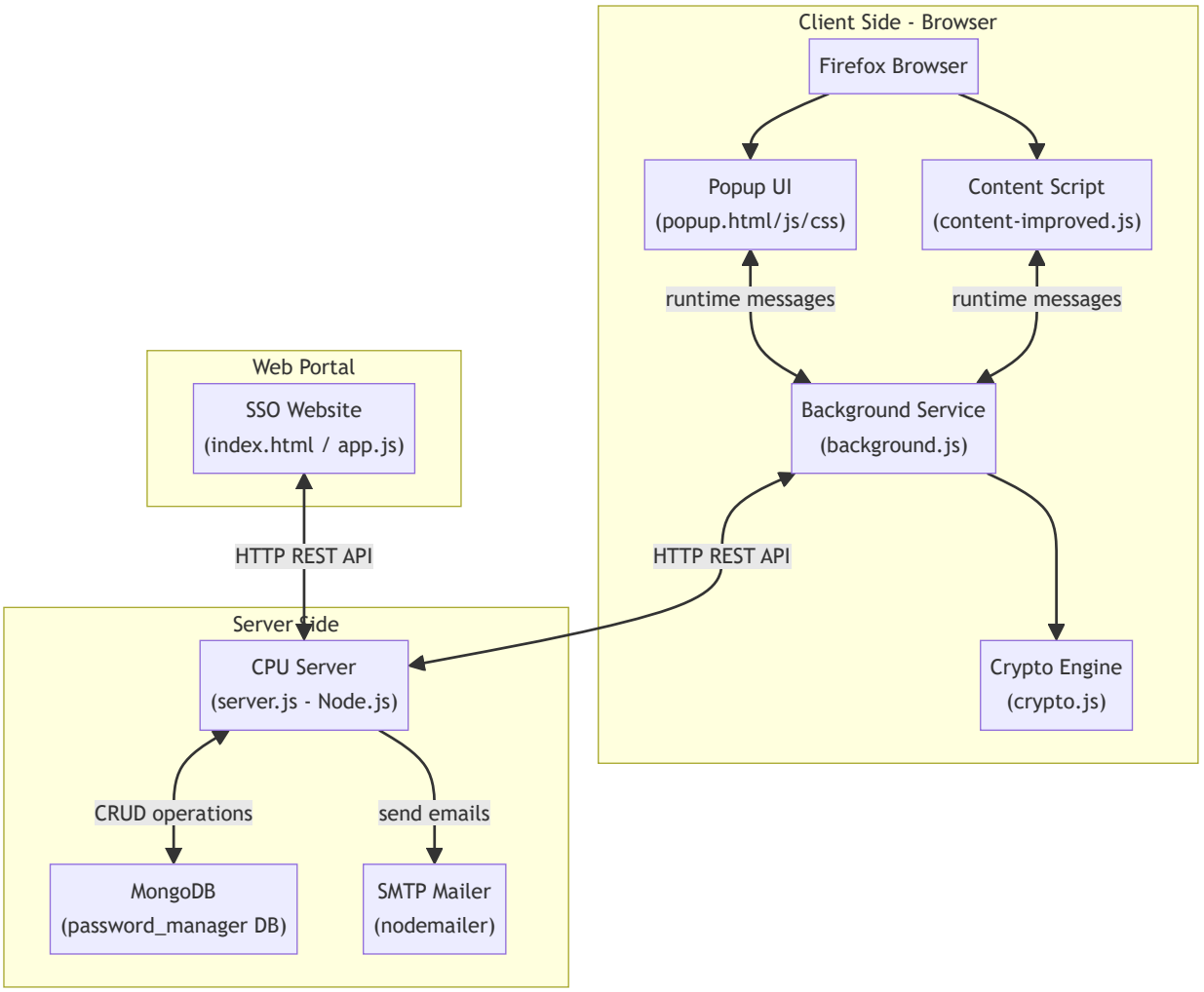
2.5 Assumptions and Dependencies

- User has Firefox browser installed
- Node.js runtime (v25.5+) is available for the backend
- MongoDB Atlas or local instance is available for production deployment
- HTTPS will be enforced in production (currently HTTP for development)

---

3. System Architecture

3.1 High-Level Architecture



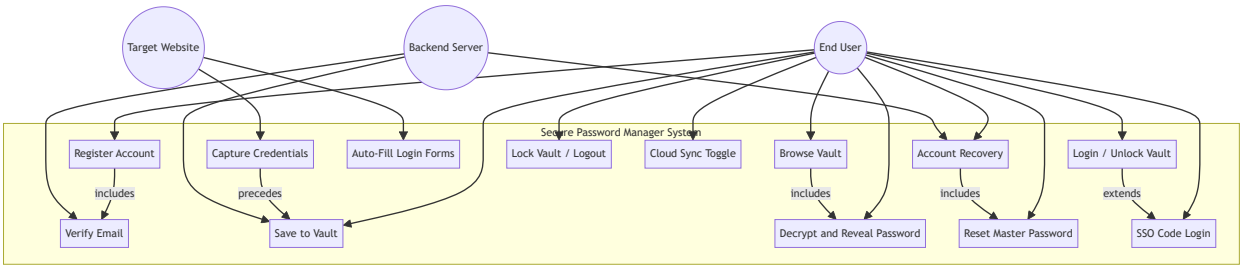
3.2 Technology Stack

Layer	Technology
Frontend (Extension)	HTML5, CSS3, Vanilla JavaScript
Frontend (Web Portal)	HTML5, CSS3, Vanilla JavaScript
Background Processing	Firefox WebExtension API (Manifest V2)
Cryptography	Web Crypto API (AES-GCM, PBKDF2, SHA-256)
Backend Server	Node.js (native http module)
Database	MongoDB (via mongodb npm driver v6.x)

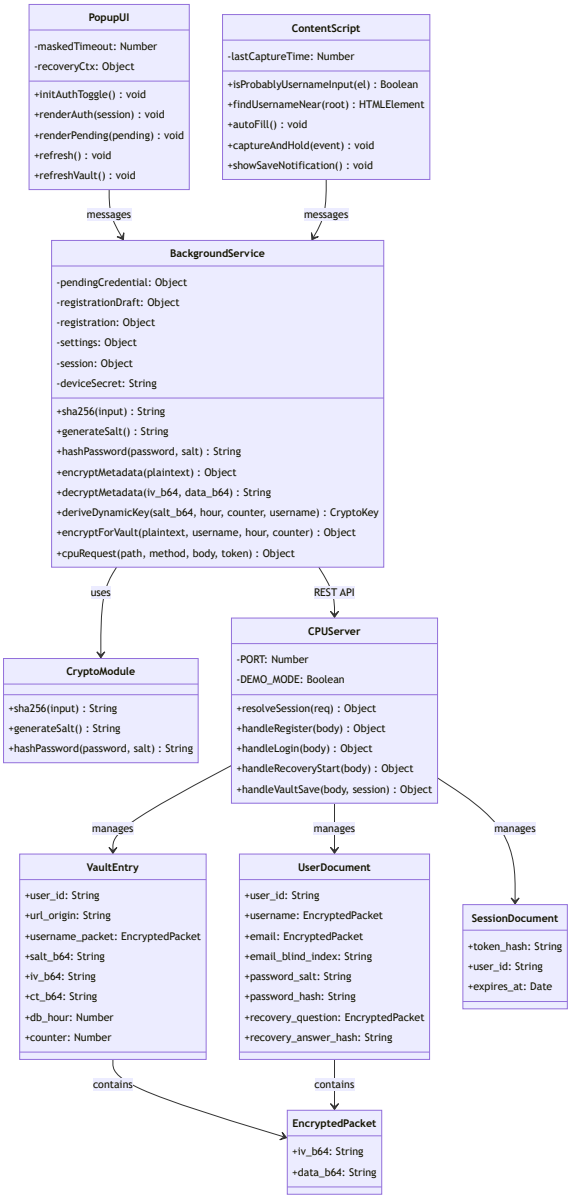
Email	Nodemailer (SMTP transport)
Configuration	dotenv

4. UML Diagrams

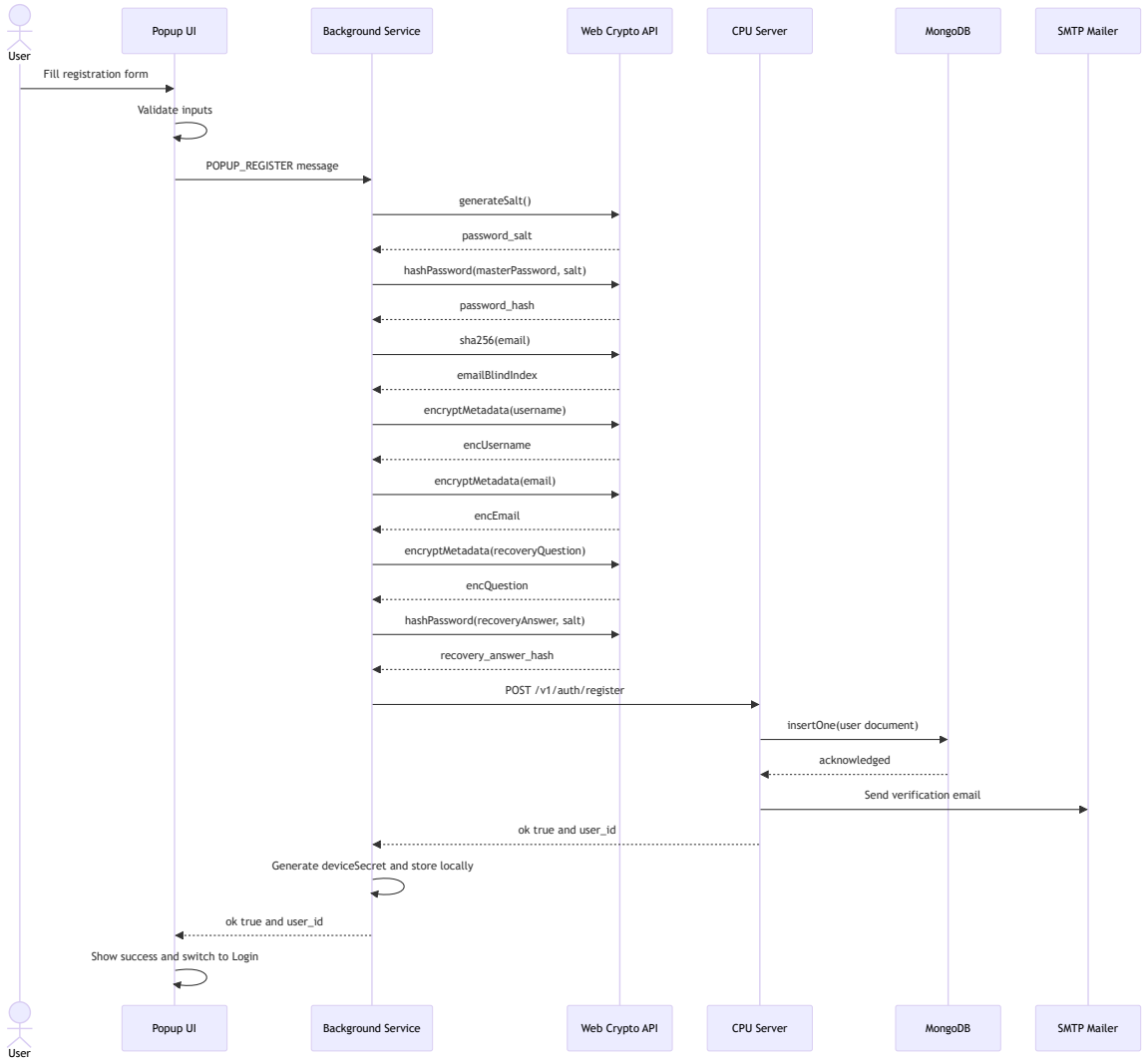
4.1 Use Case Diagram



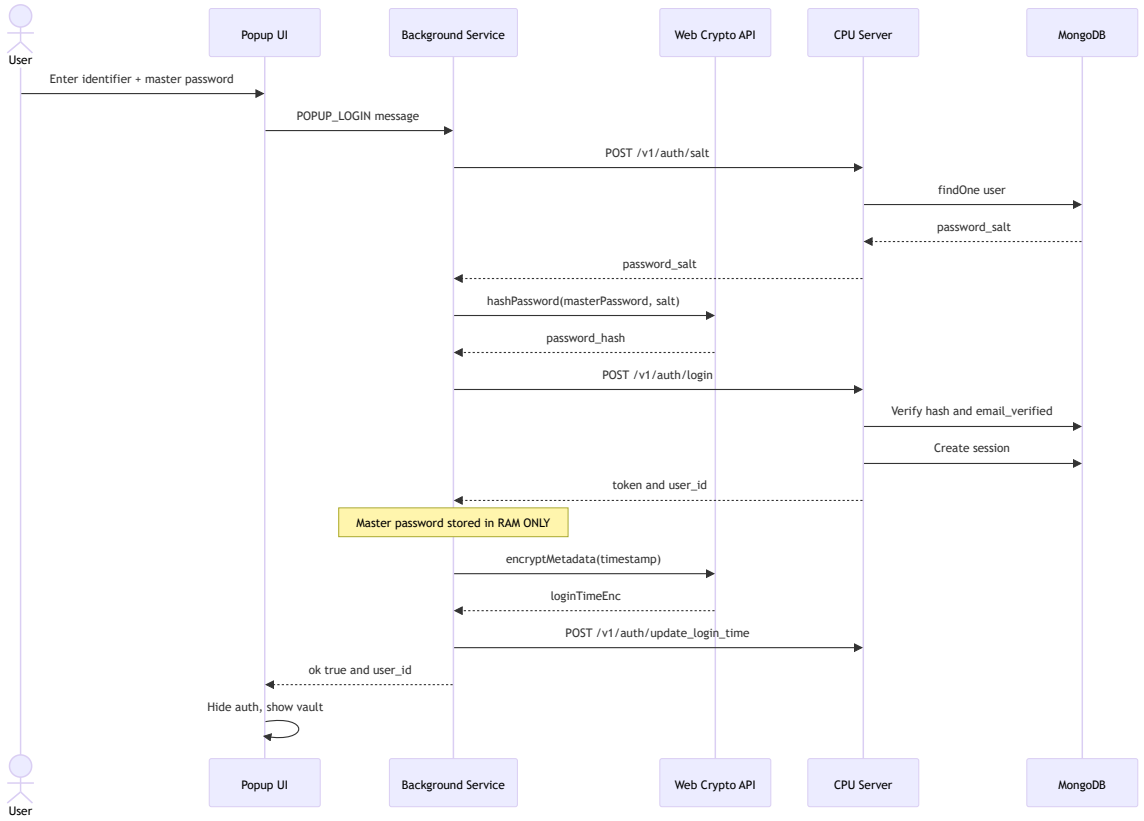
4.2 Class Diagram



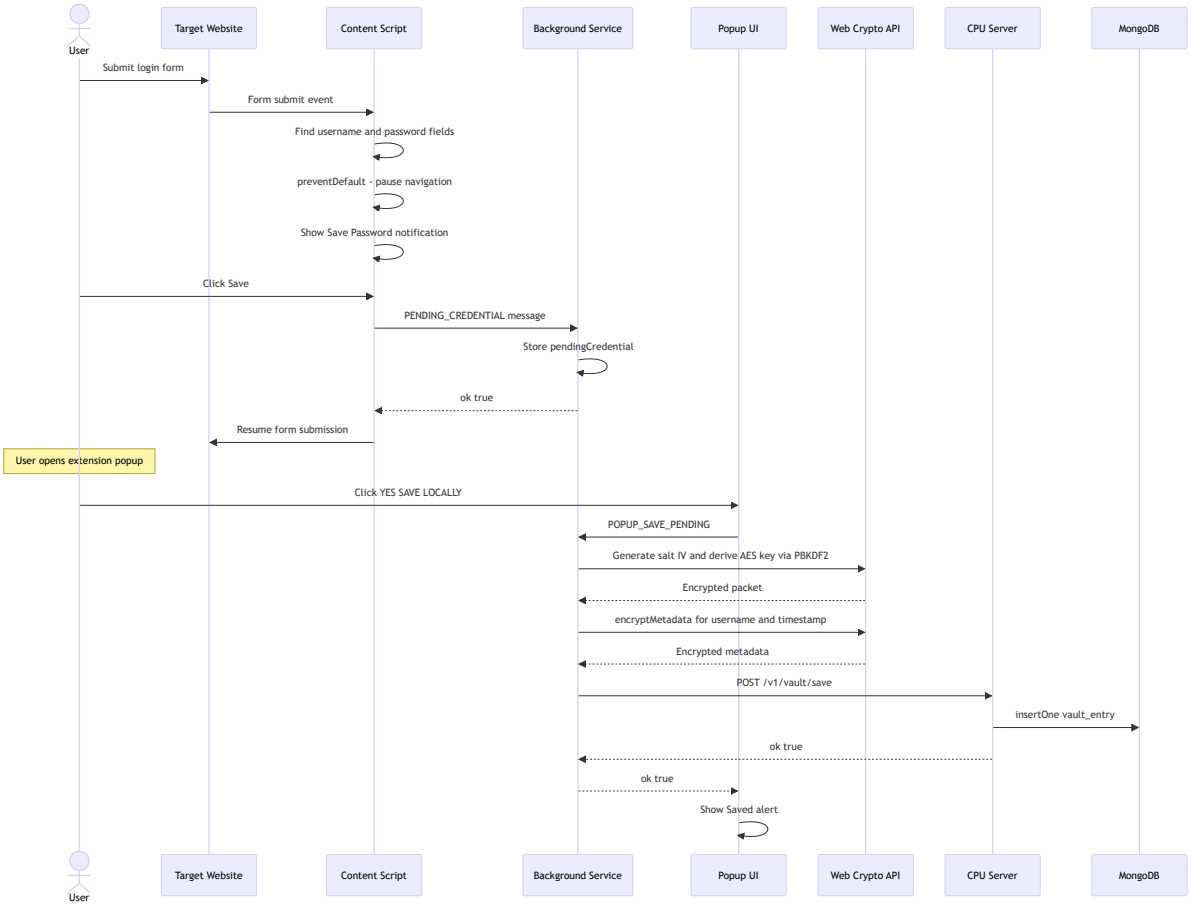
4.3 Sequence Diagram — User Registration



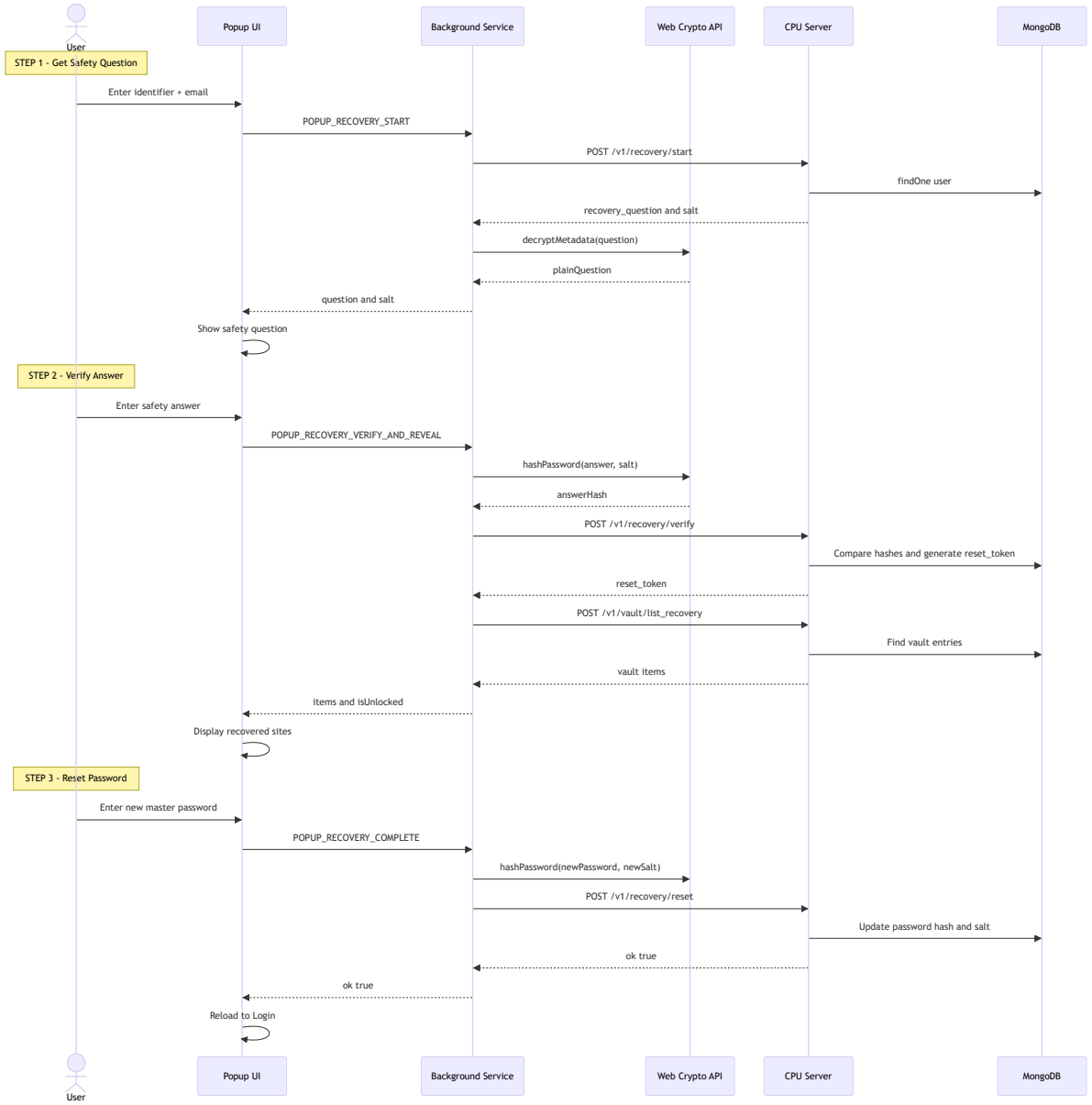
4.4 Sequence Diagram — Login & Vault Unlock



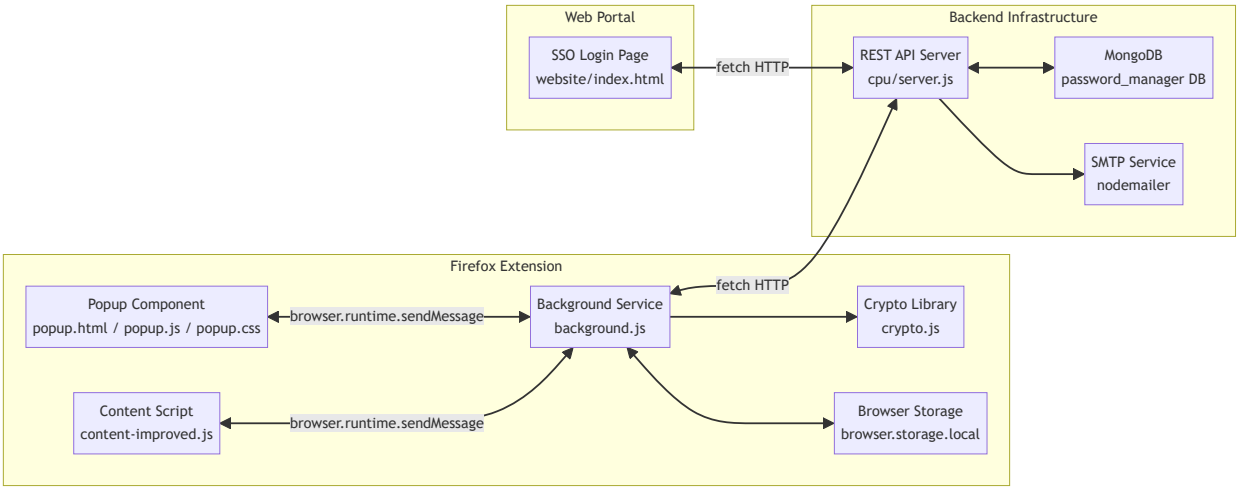
4.5 Sequence Diagram — Credential Capture & Save



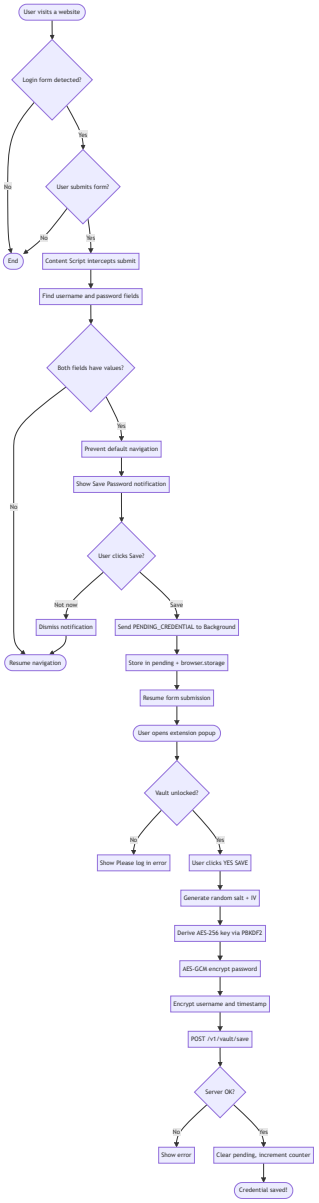
4.6 Sequence Diagram — Account Recovery



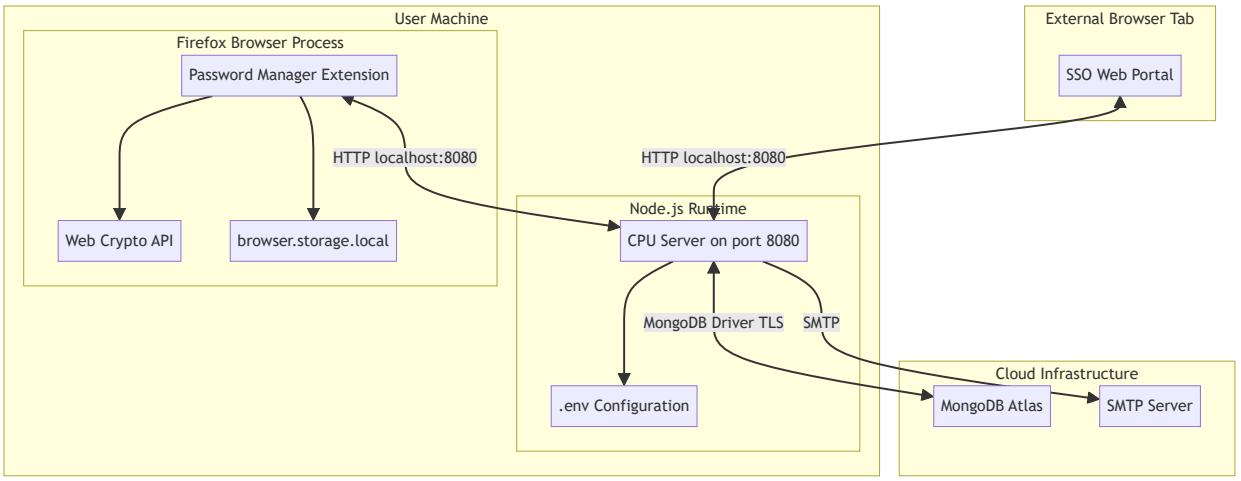
4.7 Component Diagram



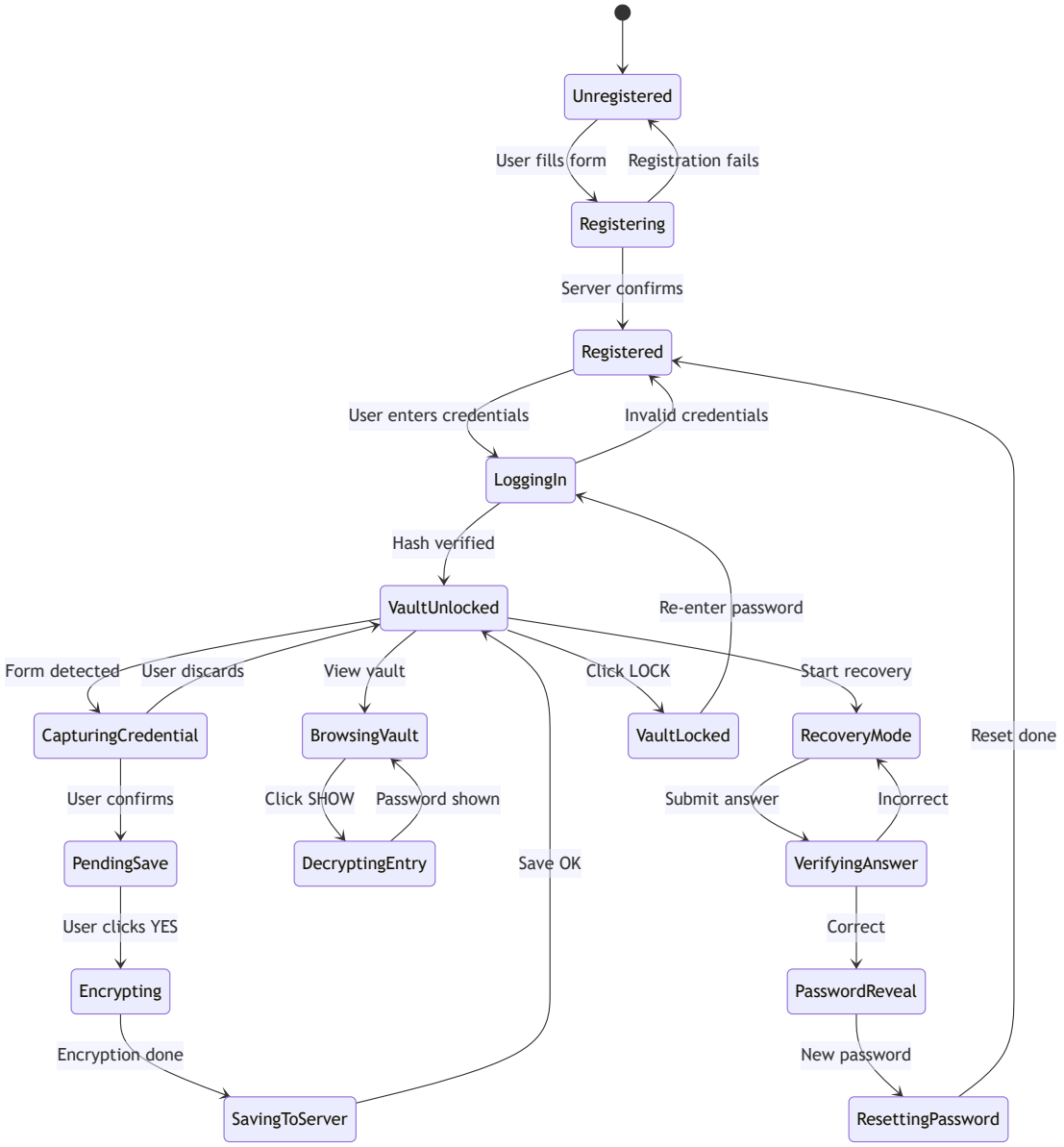
4.8 Activity Diagram — Credential Capture & Save Flow



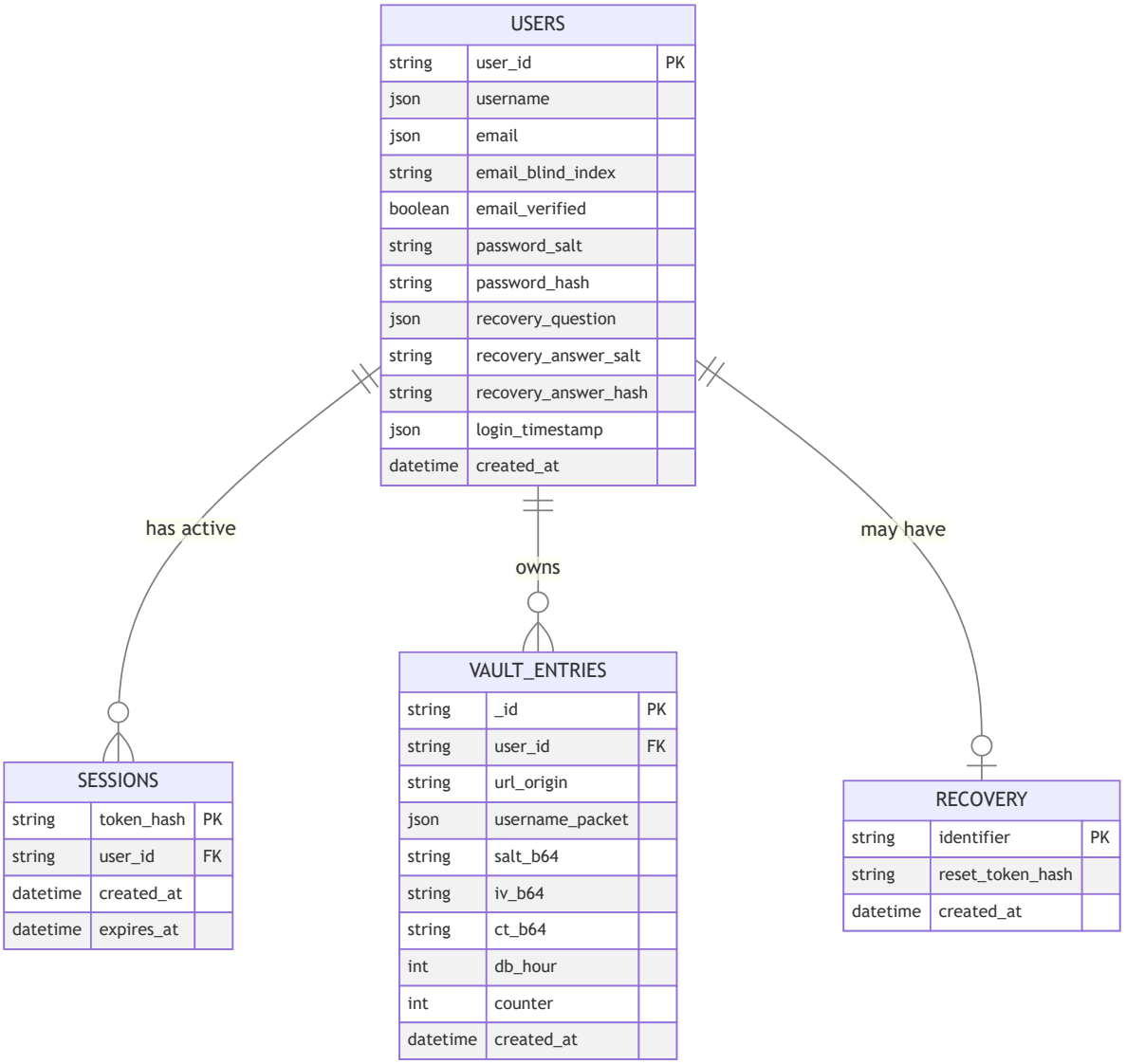
4.9 Deployment Diagram



4.10 State Machine Diagram — Vault Session Lifecycle



4.11 Entity-Relationship Diagram (Database)



5. Functional Requirements

FR-01: User Registration

Attribute	Detail
Priority	HIGH
Input	Username, Email (×2), Master Password (×2), Safety Question, Safety Answer
Validation	Email fields must match; Password fields must match
Process	1. Generate password salt → SHA-256 hash password 2. Compute email blind index 3. AES-GCM encrypt username, email, question 4. Generate recovery_answer_salt → hash answer 5. POST to /v1/auth/register 6. Server generates user_id, sends verification email 7. Generate deviceSecret → store locally
Output	Success status with user_id

FR-02: Email Verification

Attribute	Detail
Priority	MEDIUM
Input	Verification token (URL query parameter)
Process	User clicks link → Server hashes token → finds user → sets email_verified = true
Constraint	Token expires after 24 hours

FR-03: Login / Vault Unlock

Attribute	Detail
Priority	HIGH
Input	Identifier (username or email), Master Password
Process	1. Fetch password_salt from server 2. Hash password client-side 3. POST to /v1/auth/login 4. Server verifies hash + email status 5. Server creates session (6h expiry) 6. Background stores master_password in RAM

Security	Master password ONLY exists in background process memory

FR-04: Credential Capture

Attribute	Detail
Priority	HIGH
Input	Form submission event on any website
Process	1. Content script intercepts submit/click 2. Heuristic detection of username/password fields 3. Show floating notification 4. On confirm: send PENDING_CREDENTIAL to background
Constraint	5-second cooldown between captures

FR-05: Credential Save to Vault

Attribute	Detail
Priority	HIGH
Input	Pending credential (site, username, password)
Process	1. Verify session active 2. Generate random salt (16B) and IV (12B) 3. PBKDF2 derive AES-256 key (200K iterations) 4. AES-GCM encrypt password 5. Metadata-encrypt username and timestamp 6. POST to /v1/vault/save

FR-06: Vault Browse & Decrypt

Attribute	Detail
Priority	HIGH
Process	Fetch vault list → Display sites → On SHOW: re-derive key → AES-GCM decrypt

FR-07: Auto-Fill

Attribute	Detail
Priority	MEDIUM
Process	On page load: check login state → fetch vault → match origin → decrypt → fill fields

FR-08: Account Recovery

Attribute	Detail
Priority	HIGH
Process	3-step: (1) Fetch + decrypt safety question, (2) Hash answer → verify → get reset_token, (3) List vault entries

FR-09: Password Reset

Attribute	Detail
Priority	HIGH
Process	Re-verify answer → generate new salt + hash → POST to /v1/recovery/reset
Constraint	Minimum 8 characters for new password

FR-10: SSO Code Login

Attribute	Detail
Priority	LOW
Process	Web portal authenticates → issues SSO code → user enters in extension

FR-11: Session & Lock Management

Attribute	Detail
Priority	HIGH
Process	Lock: clear master_password from RAM → invalidate token → update UI

---

6. Non-Functional Requirements

NFR-01: Security

ID	Requirement
NFR-01.1	All passwords encrypted using AES-256-GCM before transmission
NFR-01.2	Key derivation uses PBKDF2 with minimum 200,000 iterations
NFR-01.3	Master password NEVER stored persistently (RAM-only during session)
NFR-01.4	Email addresses stored as blind indices (SHA-256 hashes)
NFR-01.5	All metadata encrypted with AES-GCM
NFR-01.6	Session tokens SHA-256 hashed before storage
NFR-01.7	Recovery tokens expire within 1 hour

NFR-02: Performance

ID	Requirement
NFR-02.1	Credential capture adds <200ms latency to form submission
NFR-02.2	Vault decryption completes within 500ms per entry
NFR-02.3	Server API response time under 300ms

NFR-03: Usability

ID	Requirement
NFR-03.1	Popup UI width: 420px
NFR-03.2	Passwords masked by default with explicit reveal
NFR-03.3	Save notification auto-hides after 5 seconds
NFR-03.4	Clear visual indicators for locked/unlocked states

NFR-04: Reliability

ID	Requirement
NFR-04.1	Pending credentials persist across popup close/reopen

NFR-04.2	Graceful handling of server unavailability
NFR-04.3	Demo mode provides full functionality without MongoDB

NFR-05: Compatibility

ID	Requirement
NFR-05.1	Firefox 78.0+
NFR-05.2	Node.js v25.5+
NFR-05.3	MongoDB driver v6.x

7. External Interface Requirements

7.1 User Interfaces

Interface	Description
Popup Extension	420px-wide dark-themed panel with sections for capture, auth, vault, recovery
Save Notification	Floating gradient notification (300px wide) injected into websites
SSO Web Portal	Standalone HTML page for SSO code generation

7.2 Software Interfaces

Interface	Protocol	Description
Browser ↔ Background	browser.runtime.sendMessage	Async message passing (JSON)
Background ↔ Server	HTTP REST	JSON over HTTP (localhost:8080)
Server ↔ MongoDB	MongoDB Wire Protocol	Via mongodb npm driver
Server ↔ SMTP	SMTP/TLS	Via nodemailer

7.3 API Endpoints

Method	Endpoint	Auth	Description
POST	/v1/auth/register	None	Create new user account
GET	/v1/email/verify	Token (query)	Verify email address
POST	/v1/auth/salt	None	Fetch user's password salt
POST	/v1/auth/login	None	Authenticate and get session token
POST	/v1/auth/update_login_time	Bearer	Update encrypted login timestamp
POST	/v1/vault/save	Bearer	Save encrypted credential
POST	/v1/vault/list_recovery	Reset token	List vault entries for recovery
POST	/v1/recovery/start	None	Get encrypted safety question
POST	/v1/recovery/verify	None	Verify safety answer
POST	/v1/recovery/reset	Reset token	Update master password
GET	/health	None	Server health check

8. Data Dictionary

8.1 Message Types (Extension Internal)

Message Type	Direction	Purpose
POPUP_GET_STATE	Popup → BG	Get current app state
PENDING_CREDENTIAL	Content → BG	Stage captured credentials
POPUP_DISCARD_PENDING	Popup → BG	Discard pending credentials
POPUP_REGISTER	Popup → BG	Register new account
POPUP_LOGIN	Popup → BG	Authenticate user
POPUP_LOGOUT	Popup → BG	Lock vault
POPUP_SAVE_PENDING	Popup → BG	Encrypt and save credentials
POPUP_DECRYPT_VAULT_ITEM	Popup → BG	Decrypt vault item password
POPUP_DECRYPT_RECOVERY_ITEM	Popup → BG	Decrypt metadata during recovery
POPUP_RECOVERY_START	Popup → BG	Begin recovery process
POPUP_RECOVERY_VERIFY_AND_REVEAL	Popup → BG	Verify answer and reveal vault
POPUP_RECOVERY_COMPLETE	Popup → BG	Reset master password

8.2 Encryption Schemes

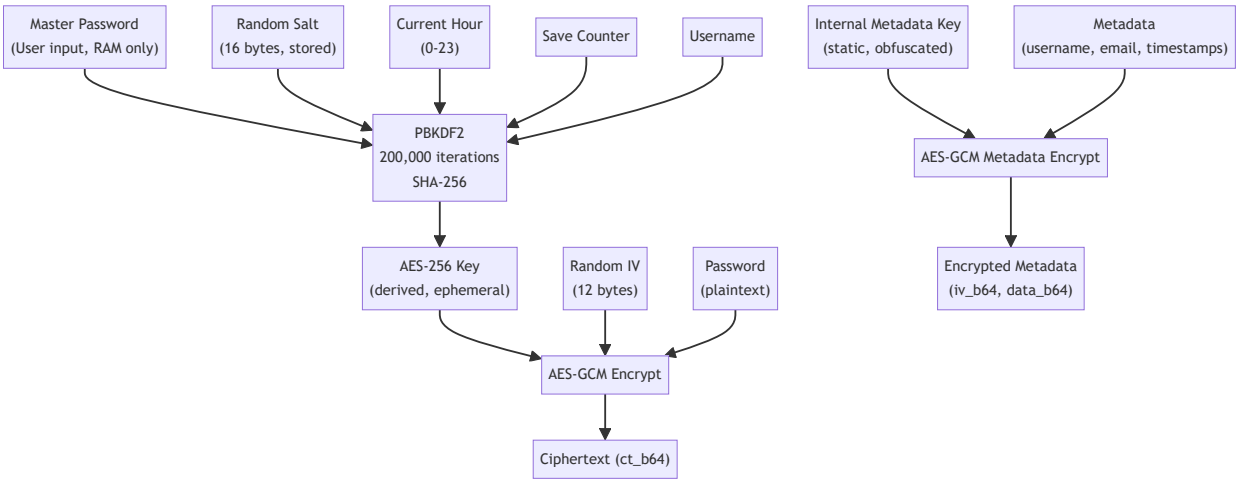
Scheme	Algorithm	Key Source	Used For
Metadata Encryption	AES-256-GCM	Static internal key (SHA-256 of hex string)	Username, email, timestamps, safety question
Vault Encryption	AES-256-GCM	PBKDF2-derived dynamic key	Stored passwords
Password Hashing	SHA-256	N/A	Master password, recovery answer
Blind Indexing	SHA-256	N/A	Email lookup without exposing plaintext

9. Security Requirements

9.1 Threat Model

Threat	Mitigation
Server database breach	Passwords encrypted client-side; server never sees plaintext
Network eavesdropping	HTTPS enforcement (production); encrypted payloads regardless
Brute-force master password	SHA-256 with random salt; PBKDF2 200K iterations
Session hijacking	Token SHA-256 hashed; 6-hour expiry
Email enumeration	Blind index prevents direct email comparison
Cross-site scripting (XSS)	Extension CSP; isolated context
RAM dump attack	Master password held temporarily; cleared on lock

9.2 Encryption Architecture



10. Appendix

10.1 File Structure

```
password_manager_2nd_draft-master/
├── manifest.json              # Extension manifest (Manifest V2)
├── package.json               # Root project dependencies
├── .gitignore
│
├── background/
│   ├── background.js          # Core logic: crypto, auth, vault, messaging
│   └── demo-log.js            # Demo mode logging utilities
│
├── content/
│   └── content-improved.js    # Credential capture & auto-fill
│
├── crypto/
│   └── crypto.js              # Shared hash/salt utilities
│
├── popup/
│   ├── popup.html             # Extension popup UI
│   ├── popup.js               # Popup interaction logic
│   └── popup.css              # Dark theme styling
│
├── cpu/
│   ├── server.js              # Node.js REST API backend
│   ├── package.json           # Server dependencies
│   ├── .env                   # Environment configuration
│   └── .env.example           # Configuration template
│
├── website/
│   ├── index.html             # SSO web portal
│   ├── app.js                 # SSO client logic
│   └── style.css              # Portal styling
│
└── icons/
    └── icon-64.png            # Extension icon
```

10.2 Environment Variables

Variable	Required	Default	Description
MONGODB_URI	Yes	—	MongoDB connection string
DB_NAME	No	password_manager	Database name
PORT	No	8080	Server port
PUBLIC_BASE_URL	No	http://localhost:8080	Public URL for email links

SMTP_HOST	No	—	SMTP server hostname
SMTP_PORT	No	587	SMTP port
SMTP_USER	No	—	SMTP username
SMTP_PASS	No	—	SMTP password
EMAIL_FROM	No	—	Sender email address

10.3 Browser Permissions

Permission	Justification
storage	Persist session, pending credentials, device secret
unlimitedStorage	Allow large vault storage
activeTab	Access current tab for content script injection
webRequest	Monitor network requests for credential detection
://localhost/	Communicate with local CPU server
http://*/*, https://*/*	Content script injection on all sites

---

Phase-1 Limitations: The current version runs in Demo Mode with in-memory mock database collections. All MongoDB operations succeed but do not persist data. Phase-2 will enable real MongoDB persistence, email verification enforcement, and production HTTPS deployment.

Document generated on: 2026-04-04 | Project: Secure Password Manager v0.1.0 | Standard: IEEE 830-1998